Project: A From-Scratch N-body Simulator

Direct summation, softening, leapfrog, Plummer initial conditions, diagnostics

astro-lab – reading notes

Goals

Understand every line of a small gravitational N-body code, know what each test verifies, and have a clear plan for making it faster (Barnes–Hut, then GPU).

1 The model

N point masses interact through Newtonian gravity. In units $G = 1$ with Plummer softening ε, the acceleration of particle i is

$$\mathbf{a}_i = G \sum_{j \neq i} \frac{m_j \, (\mathbf{r}_j - \mathbf{r}_i)}{(|\mathbf{r}_j - \mathbf{r}_i|^2 + \varepsilon^2)^{3/2}}.$$

Softening removes the singularity at close encounters. It corresponds exactly to the potential energy

$$W = -G \sum_{i<j} \frac{m_i m_j}{\sqrt{|\mathbf{r}_{ij}|^2 + \varepsilon^2}},$$

and the code uses this same W, so total energy $E = K + W$ is the quantity a symplectic integrator should conserve. Using an inconsistent potential would make the energy test meaningless.

2 Code structure

File	Role
`forces.py`	vectorised $O(N^2)$ accelerations (pairwise displacement array, no self-force)
`integrators.py`	kick–drift–kick leapfrog step and a run loop with a callback
`ics.py`	circular binary, Plummer sphere
`diagnostics.py`	kinetic, potential, total energy, momentum, virial ratio

Memory is $O(N^2)$ because the full displacement array is built; this limits the NumPy version to a few thousand particles, which is fine for learning and for testing a faster implementation against.

```
import numpy as np


def accelerations(pos, mass, G=1.0, eps=0.0):
    """Direct O(N^2) accelerations with Plummer softening eps. pos: (N,3), mass: (N,)."""
    d = pos[None, :, :] - pos[:, None, :]          # d[i,j] = r_j - r_i
    r2 = (d**2).sum(-1) + eps**2
    np.fill_diagonal(r2, np.inf)                   # no self-force
```

```
    inv_r3 = r2**-1.5
    return G * (d * (mass[None, :, None] * inv_r3[:, :, None])).sum(1)
```

3 Leapfrog

The integrator is the same kick–drift–kick scheme from Session 2, applied to all particles at once. The acceleration at the end of one step is reused as the start of the next, so each step costs one force evaluation. A fixed timestep is used; it should resolve the shortest dynamical time in the system.

```
import numpy as np
from .forces import accelerations


def leapfrog_step(pos, vel, mass, dt, acc=None, G=1.0, eps=0.0):
    """One kick-drift-kick step. Returns (pos, vel, acc) so acc can be reused."""
    if acc is None:
        acc = accelerations(pos, mass, G, eps)
    vel = vel + 0.5 * dt * acc
    pos = pos + dt * vel
    acc = accelerations(pos, mass, G, eps)
    vel = vel + 0.5 * dt * acc
    return pos, vel, acc


def run(pos, vel, mass, dt, steps, G=1.0, eps=0.0, every=1, callback=None):
    """Integrate `steps` steps; call callback(step, pos, vel) every `every` steps."""
    acc = None
    for k in range(1, steps + 1):
        pos, vel, acc = leapfrog_step(pos, vel, mass, dt, acc, G, eps)
        if callback and k % every == 0:
            callback(k, pos, vel)
    return pos, vel
```

4 Plummer sphere

The Plummer model has density and enclosed mass

$$\rho(r) = \frac{3M}{4\pi a^3}\left(1 + \frac{r^2}{a^2}\right)^{-5/2}, \qquad M(<r) = \frac{Mr^3}{(r^2+a^2)^{3/2}},$$

and potential $\Phi(r) = -GM/\sqrt{r^2+a^2}$.

Sampling positions. Draw $u \sim U(0,1)$ and set $M(<r)/M = u$. Writing $x = r^2/a^2$ gives $u^{2/3} = x/(1+x)$, so

$$r = \frac{a}{\sqrt{u^{-2/3}-1}}.$$

Directions are uniform on the sphere.

Sampling speeds. The escape speed is $v_{\rm esc} = \sqrt{2GM}\,(r^2+a^2)^{-1/4}$. With $q = v/v_{\rm esc}$, the distribution function implies q has density $g(q) \propto q^2(1-q^2)^{7/2}$, sampled by rejection. Finally the system is shifted to the centre-of-mass frame in position and velocity.

```
import numpy as np

```

```python

def circular_binary(m1=1.0, m2=1.0, sep=1.0, G=1.0):
    """Two bodies on a circular orbit, centre of mass at rest at the origin."""
    mass = np.array([m1, m2])
    M = m1 + m2
    pos = np.array([[-m2 / M * sep, 0, 0], [m1 / M * sep, 0, 0]], dtype=float)
    v_rel = np.sqrt(G * M / sep)
    vel = np.array([[0, -m2 / M * v_rel, 0], [0, m1 / M * v_rel, 0]], dtype=float
        )
    return pos, vel, mass


def plummer(n, a=1.0, M=1.0, G=1.0, seed=0):
    """Plummer sphere (Aarseth, Henon & Wielen 1974), recentred to COM frame."""
    rng = np.random.default_rng(seed)
    mass = np.full(n, M / n)

    u = rng.uniform(1e-4, 1 - 1e-4, n)
    r = a / np.sqrt(u ** (-2 / 3) - 1)
    pos = r[:, None] * _unit_vectors(rng, n)

    # speed: rejection-sample q with g(q) = q^2 (1-q^2)^3.5
    q = np.empty(n)
    i = 0
    while i < n:
        x, y = rng.uniform(0, 1), rng.uniform(0, 0.1)
        if y < x**2 * (1 - x**2) ** 3.5:
            q[i] = x
            i += 1
    v_esc = np.sqrt(2 * G * M) * (r**2 + a**2) ** -0.25
    vel = (q * v_esc)[:, None] * _unit_vectors(rng, n)

    pos -= np.average(pos, axis=0, weights=mass)
    vel -= np.average(vel, axis=0, weights=mass)
    return pos, vel, mass


def _unit_vectors(rng, n):
    z = rng.uniform(-1, 1, n)
    phi = rng.uniform(0, 2 * np.pi, n)
    s = np.sqrt(1 - z**2)
    return np.stack([s * np.cos(phi), s * np.sin(phi), z], axis=1)
```

5 What the tests verify

Test	Physical statement
`test_binary_energy_conserved`	circular binary over 10 periods, $\Delta t = 10^{-3}$: relative energy error below 10^{-6}
`test_binary_returns_to_start`	after exactly one period positions return to within 10^{-5} (checks the orbital frequency)
`test_momentum_conserved_plummer`	pairwise forces are equal and opposite, so total momentum is constant to round-off
`test_newton_third_law`	$\sum_i m_i \mathbf{a}_i = 0$ for random masses and positions
`test_plummer_roughly_virialised`	$2K/\lvert W\rvert$ within 0.8–1.2 for $N = 2000$

Example run (`run_plummer.py 256 300`, $\varepsilon = 0.05$, $\Delta t = 0.01$): initial virial ratio 1.109, final 1.038, relative energy change -1.9×10^{-7}. The initial virial ratio is not exactly 1 because $N = 256$

is small (sampling noise) and the sampling ignores softening.

6 Roadmap: making it fast

1. **Barnes–Hut tree.** Group distant particles into a tree of cells; if a cell of size s at distance d satisfies $s/d < \theta$ (typically $\theta \approx 0.5$–0.7), treat it as a single point mass at its centre of mass. Cost drops from $O(N^2)$ to $O(N \log N)$ at the price of a controlled force error. Validate against the direct sum on the same particle set and plot force error versus θ.
2. **GPU forces.** The all-pairs kernel is a natural fit for a compute shader: one thread per particle, tile positions through shared memory. Benchmark against the NumPy version for $N = 10^3$ to 10^5 and plot time versus N on log–log axes.
3. **Timestepping.** Individual or block timesteps help when particles have very different dynamical times; they need care to retain good energy behaviour.
4. **External validation.** Reproduce the figure-eight three-body orbit and compare with REBOUND.

7 Exercises

1. Plot energy error versus Δt for the Plummer run; confirm it scales as Δt^2 for a fixed duration.
2. Plot the Lagrangian radii (containing 10%, 50%, 90% of the mass) over time. Do they stay constant for a Plummer sphere in equilibrium?
3. Remove softening ($\varepsilon = 0$) and find a close encounter that ruins energy conservation.
4. Estimate the relaxation time and explain why a collisionless approximation is appropriate for a galaxy but not for a globular cluster over many Gyr.

References

- Aarseth, Henon, Wielen (1974), *A&A* 37, 183 (Plummer initial conditions)
- Barnes & Hut (1986), *Nature* 324, 446
- Binney & Tremaine, *Galactic Dynamics*, ch. 2
- Nyland, Harris, Prins, "Fast N-body Simulation with CUDA", *GPU Gems 3*, ch. 31